

به نام خدا



دانشکده فنی و مهندسی دانشگاه الزهرا

گروه مهندسی برق

گزارشکار تمرین ۴

تکلیف سری چهارم Interrupt ,Timer

طراحی بر اساس ریزپردازنده

استاد درس: سرکار خانم دکتر امینی

دستیاران آموزشی: تمنا امرجی، کیانا احمدپور و ریحانه شاکری

دانشجویان:

نگار سیاه منصور ۴۰۲۱۵۳۰۱۰۰۸

محدثه اسمعیل زاده ۴۰۲۱۵۳۰۲۰۰۱

۱- برنامه‌ای طراحی کنید که یک ساعت دیجیتال به فرمت HH:MM را روی سون‌سگمنت ۴ رقمی نمایش دهد. ساعت باید از مقدار ۰۰:۰۰ شروع شده و زمان را به صورت خودکار افزایش دهد.

نکات الزامی:

۱. استفاده از توابع Delay و هرگونه حلقه تأخیری برای تولید زمان مجاز نیست.
۲. نمایش ارقام روی سون‌سگمنت ۴ رقمی باید به روش Multiplexing پیاده‌سازی شود.
۳. قالب ساعت ۲۴ ساعته می‌باشد.
۴. نرخ به‌روزرسانی سون‌سگمنت باید به گونه‌ای انتخاب شود که چشمک‌زدن یا خاموش و روشن شدن ارقام قابل مشاهده نباشد.

مقدمه

در این سوال هدف طراحی یک ساعت دیجیتال با فرمت دقیقه و ثانیه، یعنی MM:SS، روی یک سون‌سگمنت چهار رقمی بود. در صورت تمرین تأکید شده بود که برای تولید زمان نباید از تابع‌های تأخیری مانند Delay یا حلقه‌های تأخیری استفاده شود. همچنین نمایش چهار رقم باید به روش Multiplexing انجام شود تا با استفاده از تعداد پایه کمتر، چهار رقم سون‌سگمنت کنترل شوند.

در روش Multiplexing، در هر لحظه فقط یکی از چهار رقم سون‌سگمنت روشن می‌شود، اما این کار با سرعت زیاد بین رقم‌ها تکرار می‌شود. به دلیل سرعت بالای روشن و خاموش شدن رقم‌ها، چشم انسان همه رقم‌ها را هم‌زمان روشن می‌بیند. در این پروژه برای تولید زمان و refresh کردن نمایشگر از تایمر و وقفه استفاده شد و هیچ تابع Delay در کد نهایی استفاده نشد.

هدف پروژه

هدف‌های اصلی این پروژه عبارت بودند از:

۱. طراحی ساعت دیجیتال با فرمت MM:SS.
۲. شروع شمارش از مقدار 00:00.
۳. افزایش خودکار ثانیه‌ها و دقیقه‌ها.
۴. استفاده از سون‌سگمنت چهار رقمی 7SEG-MPX4-CC.
۵. پیاده‌سازی نمایش به روش Multiplexing.

۶. استفاده از تایمر و وقفه برای تولید زمان.

۷. حذف کامل HAL از کد نهایی و استفاده از برنامه‌نویسی رجیستری.

۸. انجام شبیه‌سازی در نرم‌افزار Proteus.

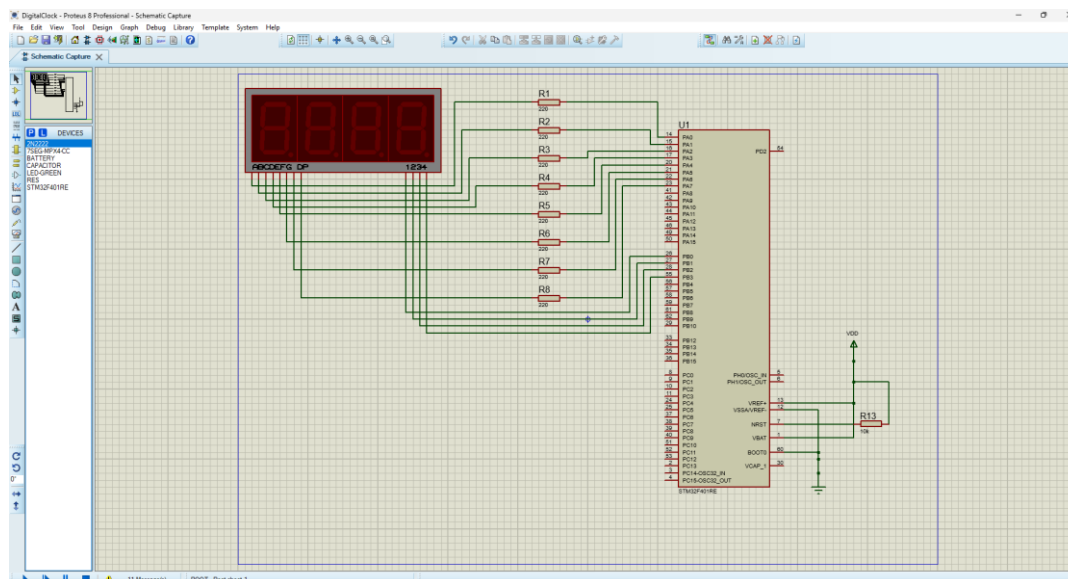
۹. ساخت پروژه و فایل HEX با STM32CubeIDE و STM32CubeMX.

قطعات استفاده‌شده در Proteus

برای شبیه‌سازی مدار از قطعات زیر استفاده شد:

نام قطعه	توضیح
STM32F401RE	میکروکنترلر اصلی پروژه
7SEG-MPX4-CC	سون سگمنت چهار رقمی کاتد مشترک
330Ω	مقاومت برای محدود کردن جریان سگمنت‌ها
10kΩ	مقاومت برای پایه Reset میکروکنترلر
3.3V	منبع تغذیه میکروکنترلر
GND	زمین مدار

در این مدار از ترانزیستور استفاده نشد و پایه‌های انتخاب رقم‌های سون سگمنت به صورت مستقیم به پایه‌های میکروکنترلر وصل شدند.



توضیح مدار Proteus

در مدار Proteus از میکروکنترلر STM32F401RE استفاده شد. سون سگمنت انتخاب شده از نوع 7SEG-MPX4-CC است. عبارت CC یعنی Common Cathode، بنابراین سگمنت‌ها با یک منطقی روشن می‌شوند و پایه‌های انتخاب رقم‌ها با صفر منطقی فعال می‌شوند.

اتصالات سگمنت‌ها به این صورت انجام شد:

اتصال به سون سگمنت	پایه میکروکنترلر
سگمنت A از طریق مقاومت 330Ω	PA0
سگمنت B از طریق مقاومت 330Ω	PA1
سگمنت C از طریق مقاومت 330Ω	PA2
سگمنت D از طریق مقاومت 330Ω	PA3
سگمنت E از طریق مقاومت 330Ω	PA4
سگمنت F از طریق مقاومت 330Ω	PA5
سگمنت G از طریق مقاومت 330Ω	PA6
پایه DP از طریق مقاومت 330Ω	PA7

پایه‌های انتخاب رقم‌های سون سگمنت به این صورت متصل شدند:

اتصال	پایه میکروکنترلر
رقم اول سون سگمنت	PB0
رقم دوم سون سگمنت	PB1
رقم سوم سون سگمنت	PB2
رقم چهارم سون سگمنت	PB3

چون سون سگمنت کاتد مشترک است، پایه‌های رقم‌ها Active Low هستند. یعنی اگر پایه PBx صفر شود، رقم مربوط به آن روشن می‌شود و اگر یک شود، آن رقم خاموش می‌شود.

برای تغذیه و راه‌اندازی میکروکنترلر نیز اتصالات زیر انجام شد:

- پایه‌های VDD به $3.3V$ وصل شدند.
- پایه‌های VSS به زمین وصل شدند.
- پایه $VREF+$ به $3.3V$ وصل شد.

- پایه -VSSA/VREF به زمین وصل شد.
- پایه BOOT0 به زمین وصل شد تا میکرو از حافظه Flash اجرا شود.
- پایه NRST با مقاومت $10k\Omega$ به $3.3V$ وصل شد.

تنظیمات انجام شده در STM32CubeMX

ابتدا پروژه در STM32CubeMX ساخته شد. میکروکنترلر انتخاب شده:

STM32F401RETx

بود. این مدل با قطعه‌ای که در Proteus استفاده شد، یعنی STM32F401RE، هماهنگ است.

در بخش Pinout، پایه‌های مورد نیاز برای سون سگمنت به صورت خروجی دیجیتال تنظیم شدند. پایه‌های PA0 تا PA7 برای کنترل سگمنت‌ها و پایه‌های PB0 تا PB3 برای کنترل رقم‌ها استفاده شدند.

تنظیمات GPIO به شکل زیر انجام شد:

پایه‌های سگمنت‌ها

PA0 تا PA7 به صورت GPIO_Output تنظیم شدند. مقدار اولیه آن‌ها Low در نظر گرفته شد، چون در شروع برنامه باید همه سگمنت‌ها خاموش باشند.

تنظیمات این پایه‌ها:

- Mode: GPIO_Output
- Output Type: Push Pull
- Pull-up/Pull-down: No Pull
- Output Speed: Low
- Initial Level: Low

پایه‌های انتخاب رقم‌ها

PB0 تا PB3 نیز به صورت GPIO_Output تنظیم شدند. مقدار اولیه این پایه‌ها High در نظر گرفته شد، چون رقم‌های سون سگمنت کاتد مشترک هستند و با صفر فعال می‌شوند.

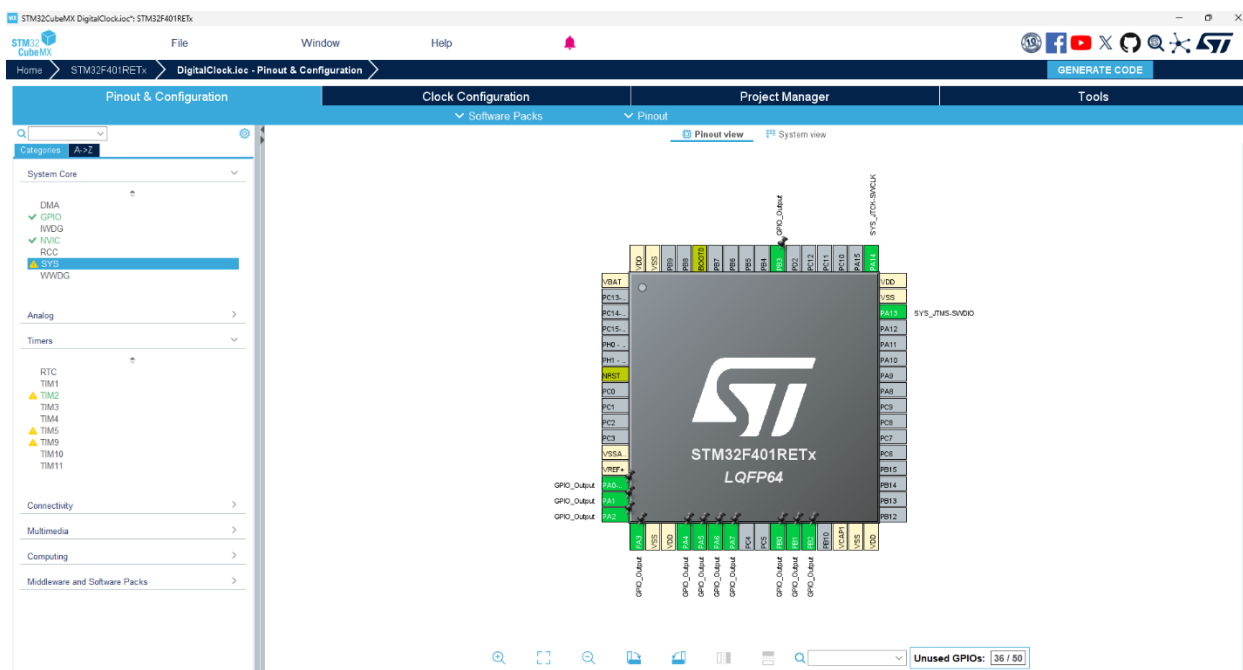
تنظیمات این پایه‌ها:

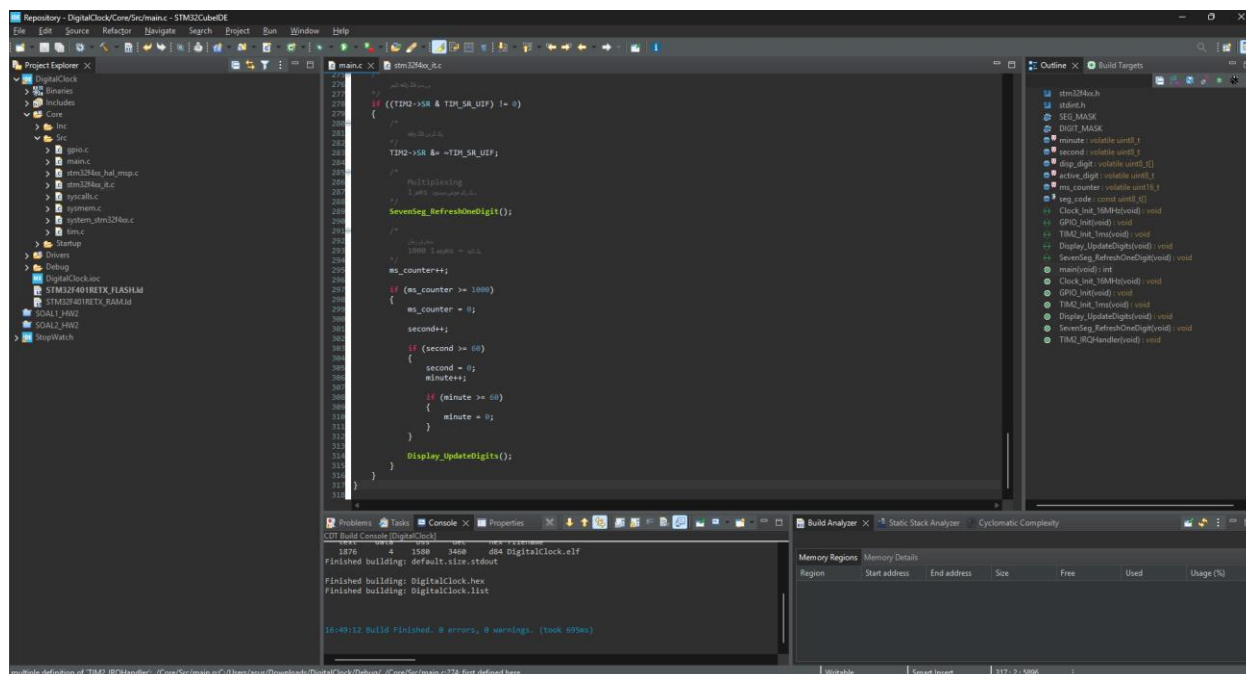
- Mode: GPIO_Output

- Output Type: Push Pull
- Pull-up/Pull-down: No Pull
- Output Speed: Low
- Initial Level: High

در بخش SYS، گزینه Debug روی حالت Serial Wire قرار داده شد تا امکان پروگرام کردن میکرو وجود داشته باشد و همزمان پایه‌های مورد نیاز مدار آزاد باشند.

در بخش Clock Configuration، در ابتدا کلاک سیستم در CubeMX روی ۸۴MHz تنظیم شده بود، اما چون در نسخه نهایی برنامه از کد کاملاً رجیستری و بدون HAL استفاده شد، راه‌اندازی کلاک مستقیماً داخل کد انجام گرفت. در کد نهایی از کلاک داخلی HSI با فرکانس ۱۶MHz استفاده شد تا شبیه‌سازی در Proteus ساده‌تر و پایدارتر باشد. بنابراین در Proteus نیز مقدار Clock Frequency میکرو روی ۱۶MHz تنظیم شد.





حذف HAL و اصلاح فایل ها

در ابتدا پروژه با CubeMX ساخته شده بود، بنابراین فایل های HAL و فایل های پیش فرض CubeMX در پروژه وجود داشتند. اما طبق خواسته پروژه، کد نهایی نباید با HAL نوشته می شد. بنابراین برنامه اصلی داخل فایل main.c به صورت رجیستری نوشته شد.

در کد نهایی از هیچ کدام از موارد زیر استفاده نشد:

- HAL_Init
- HAL_Delay
- HAL_GPIO_WritePin
- HAL_TIM_Base_Start_IT
- HAL_TIM_IRQHandler

برای کنترل GPIO ها از رجیسترهای GPIOA و GPIOB استفاده شد. برای فعال کردن کلاک پورت ها از رجیستر RCC-AHB1ENR استفاده شد. برای تنظیم تایمر نیز از رجیسترهای TIM2->PSC، TIM2->ARR، TIM2->DIER و TIM2->CR1 استفاده شد.

بعد از نوشتن کد رجیستری، یک خطای Build ایجاد شد که مربوط به تعریف تکراری تابع وقفه TIM2_IRQHandler بود. علت این خطا این بود که CubeMX خودش داخل فایل stm32f4xx_it.c یک تابع TIM2_IRQHandler ساخته بود، در حالی که همین تابع در فایل main.c هم نوشته شده بود.

برای رفع این مشکل، تابع TIM2_IRQHandler داخل فایل زیر حذف یا کامنت شد:

Core/Src/stm32f4xx_it.c

تابعی که حذف یا کامنت شد، به صورت کلی این شکل را داشت:

```
void TIM2_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim2);
}
```

با حذف این تابع از فایل stm32f4xx_it.c، فقط تابع وقفه‌ای که در main.c نوشته شده بود باقی ماند. بعد از این اصلاح، پروژه بدون خطا و بدون هشدار Build شد.

در نتیجه فایل‌های اصلی که در پروژه تغییر داده شدند عبارت‌اند از:

۱. Core/Src/main.c

کل کد اصلی پروژه به صورت رجیستری و بدون HAL در این فایل نوشته شد.

۲. Core/Src/stm32f4xx_it.c

تابع پیش‌فرض TIM2_IRQHandler که توسط CubeMX ساخته شده بود، حذف یا کامنت شد تا با تابع وقفه داخل main.c تداخل نداشته باشد.

فایل‌های HAL به صورت فیزیکی از پروژه پاک نشدند، اما در کد نهایی از آن‌ها استفاده نشد.

توضیح عملکرد کد

در ابتدای برنامه، فایل هدر stm32f4xx.h قرار داده شد تا بتوان به رجیسترهای میکروکنترلر دسترسی مستقیم داشت.

در برنامه یک آرایه برای کدهای سون‌سگمنت تعریف شد. این آرایه مشخص می‌کند برای نمایش هر عدد از ۰ تا ۹، کدام سگمنت‌ها باید روشن شوند. چون سون‌سگمنت از نوع کاتد مشترک است، روشن شدن سگمنت‌ها با مقدار ۱ انجام می‌شود.

متغیرهای اصلی برنامه عبارت‌اند از:

- minute برای نگهداری مقدار دقیقه
- second برای نگهداری مقدار ثانیه
- disp_digit[4] برای نگهداری چهار رقم قابل نمایش
- active_digit برای مشخص کردن اینکه در هر لحظه کدام رقم فعال شود

• ms_counter برای شمارش وقفه‌های یک میلی‌ثانیه‌ای

تابع Clock_Init_16MHz کلاک داخلی HSI را فعال می‌کند و سیستم را روی کلاک ۱۶ MHz قرار می‌دهد.

تابع GPIO_Init پایه‌های PA0 تا PA7 و PB0 تا PB3 را به عنوان خروجی تنظیم می‌کند. در این تابع ابتدا کلاک پورت‌های A و B فعال می‌شود، سپس حالت پایه‌ها در رجیستر MODER تنظیم می‌شود. همچنین خروجی‌ها به صورت Push Pull و بدون Pull-up/Pull-down قرار داده می‌شوند.

تابع TIM2_Init_1ms تایمر TIM2 را تنظیم می‌کند. این تابع کلاک تایمر را فعال کرده، مقدار Prescaler و ARR را تنظیم می‌کند، وقفه Update تایمر را فعال می‌کند و در نهایت تایمر را روشن می‌کند.

تابع Display_UpdateDigits مقدار دقیقه و ثانیه را به چهار رقم جدا تبدیل می‌کند. برای مثال اگر زمان برابر 02:8 باشد، مقدار نمایشگر به صورت 00.28 یا 00:28 دیده می‌شود.

تابع SevenSeg_RefreshOneDigit وظیفه Multiplexing را انجام می‌دهد. در این تابع ابتدا همه رقم‌ها خاموش می‌شوند، سپس الگوی سگمنت‌های رقم فعلی روی پورت A قرار می‌گیرد و بعد فقط همان رقم از طریق پورت B فعال می‌شود. در هر بار اجرای این تابع، فقط یک رقم روشن می‌شود و در اجرای بعدی رقم بعدی روشن می‌شود.

تابع TIM2_IRQHandler تابع وقفه تایمر است. هر بار که وقفه TIM2 اتفاق می‌افتد، ابتدا فلگ وقفه پاک می‌شود، سپس تابع refresh سون سگمنت اجرا می‌شود. بعد از آن شمارنده میلی‌ثانیه افزایش پیدا می‌کند. وقتی مقدار شمارنده به ۱۰۰۰ برسد، یعنی یک ثانیه گذشته است. در این حالت مقدار ثانیه یک واحد افزایش پیدا می‌کند. اگر ثانیه به ۶۰ برسد، صفر شده و دقیقه یک واحد زیاد می‌شود. اگر دقیقه هم به ۶۰ برسد، دوباره صفر می‌شود.

روش Multiplexing در پروژه

در این پروژه چون از سون سگمنت چهار رقمی استفاده شد، نمی‌توان همه رقم‌ها را به صورت مستقل و هم‌زمان کنترل کرد. بنابراین پایه‌های سگمنت‌ها بین هر چهار رقم مشترک هستند و فقط پایه‌های انتخاب رقم جدا هستند.

در هر وقفه تایمر، فقط یکی از رقم‌ها فعال می‌شود. چون وقفه تایمر هر ۱ میلی‌ثانیه رخ می‌دهد، هر رقم به ترتیب روشن می‌شود:

- وقفه اول: رقم اول
- وقفه دوم: رقم دوم
- وقفه سوم: رقم سوم
- وقفه چهارم: رقم چهارم
- وقفه پنجم: دوباره رقم اول

بنابراین کل نمایشگر هر ۴ میلی ثانیه یک بار کامل refresh می شود. فرکانس refresh کل نمایشگر برابر است با:

$$1 / 0.004 = 250\text{Hz}$$

این مقدار برای چشم انسان بسیار سریع است، بنابراین چشمک زدن رقم ها دیده نمی شود و نمایشگر به صورت پیوسته روشن دیده می شود.

نمایش جداکننده بین دقیقه و ثانیه

در قطعه 7SEG-MPX4-CC علامت دو نقطه واقعی بین رقم دوم و سوم وجود ندارد. به همین دلیل برای جدا کردن دقیقه و ثانیه از نقطه اعشار رقم دوم استفاده شد. به همین خاطر خروجی در شبیه سازی به شکل زیر دیده می شود:

00.28

که در واقع همان مفهوم 00:28 را دارد. یعنی دو رقم سمت چپ دقیقه و دو رقم سمت راست ثانیه هستند.

نتیجه شبیه سازی

بعد از اصلاح کد و حذف تابع تکراری وقفه، پروژه با موفقیت Build شد و در خروجی Build عبارت زیر دیده شد:

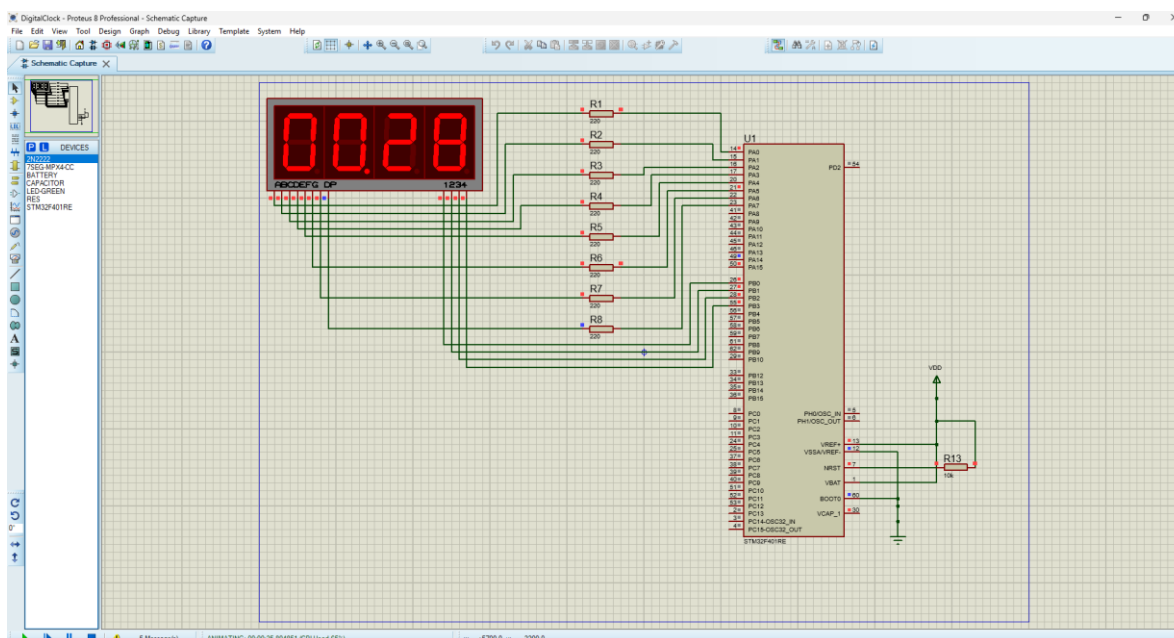
Build Finished. 0 errors, 0 warnings

سپس فایل HEX ساخته شده به میکروکنترلر در Proteus داده شد. در تنظیمات میکرو، مقدار Clock Frequency روی

16MHz قرار گرفت. بعد از اجرای شبیه سازی، سون سگمنت روشن شد و مقدار زمان روی نمایشگر دیده شد. در تصویر

شبیه سازی مقدار 00.28 نمایش داده شده است که نشان می دهد تایمر، وقفه و Multiplexing به درستی کار می کنند و

شمارش ثانیه ها انجام می شود.



مشکلاتی که هنگام انجام پروژه پیش آمد

در ابتدا نمایشگر کاملاً خاموش بود. بررسی مدار و کد نشان داد که چند دلیل ممکن است باعث این اتفاق شود، از جمله فعال نبودن وقفه تایمر، اشتباه بودن فایل HEX، یا وجود HAL در تابع وقفه.

بعد از بررسی Build، مشخص شد تابع TIM2_IRQHandler در دو فایل تعریف شده است؛ یک بار در main.c و یک بار در stm32f4xx_it.c. به همین دلیل خطای multiple definition of TIM2_IRQHandler ایجاد شد. برای رفع این خطا، تابع وقفه داخل stm32f4xx_it.c حذف یا کامنت شد و تابع وقفه رجیستری داخل main.c باقی ماند.

همچنین چون هدف پروژه استفاده نکردن از HAL بود، تمام توابع HAL از کد اصلی حذف شدند و تنظیمات GPIO، Clock و Timer مستقیماً با رجیسترها انجام شد.

بعد از این اصلاحات، برنامه بدون خطا Build شد و سون سگمنت در Proteus روشن شد.

جمع بندی تنظیمات نهایی

تنظیمات نهایی پروژه به صورت زیر است:

تنظیم نهایی	بخش
STM32F401RE	میکروکنترلر
STM32CubeMX	نرم افزار تولید پروژه
STM32CubeIDE	محیط برنامه نویسی
Proteus 8 Professional	نرم افزار شبیه سازی
7SEG-MPX4-CC	نمایشگر
Multiplexing	روش نمایش
TIM2	تایمر استفاده شده
1 میلی ثانیه	زمان هر وقفه
HSI داخلی ۱۶MHz	کلاک نهایی در کد
1599	مقدار Prescaler
9	مقدار ARR
PA0 تا PA7	پایه های سگمنت
PB0 تا PB3	پایه های انتخاب رقم

نوع کد	رجیستری، بدون HAL
استفاده از Delay	ندارد

نتیجه‌گیری

در این پروژه یک ساعت دیجیتال با فرمت دقیقه و ثانیه روی سون‌سگمنت چهار رقمی طراحی و شبیه‌سازی شد. برای تولید زمان از تایمر TIM2 و وقفه استفاده شد و هیچ تابع Delay یا حلقه تأخیری برای شمارش زمان به کار نرفت. نمایش ارقام به روش Multiplexing انجام شد و با انتخاب زمان وقفه ۱ میلی‌ثانیه، نرخ refresh نمایشگر به حدود ۲۵۰ Hz رسید؛ بنابراین چشمک زدن رقم‌ها قابل مشاهده نبود.

در نسخه نهایی کد، از HAL استفاده نشد و تمام تنظیمات GPIO، تایمر، کلاک و وقفه به صورت مستقیم با رجیسترهای میکروکنترلر انجام شد. همچنین خطای مربوط به تعریف تکراری تابع وقفه با حذف تابع TIM2_IRQHandler از فایل stm32f4xx_it.c برطرف شد. در نهایت برنامه بدون خطا Build شد و در Proteus خروجی ساعت روی سون‌سگمنت نمایش داده شد.

این پروژه نشان داد که با استفاده از تایمر و وقفه می‌توان بدون استفاده از Delay، یک سیستم زمان‌بندی دقیق طراحی کرد و با روش Multiplexing چند رقم سون‌سگمنت را با تعداد پایه کمتر کنترل نمود.

۲- با استفاده از تایمر و وقفه میکرو، کرونومتر طراحی کنید. یکی از کلیدهای فشاری موجود بر روی برد کلید start/pause کرونومتر شما خواهد بود که از طریق وقفه خارجی خوانده می‌شود با فشردن این کلید کرونومتر شما با دقت میلی ثانیه شروع به شمارش خواهد کرد و با فشردن مجدد این کلید، شمارش متوقف خواهد شد. عدد کرونومتر را پردازش کنید، در صورتی که زوج بود، بازر بوق کوتاهی بزند. همزمان یک led چشمک‌زن داریم که سرعت آن با فشردن شدن یک کلید بر روی برد افزایش می‌یابد. تاخیر چشمک این led با ۳۰۰ ms شروع و با هر بار فشردن شدن کلید مقدار تاخیر این چشمک نصف می‌شود تا زمانی که به ۳۷۵ ms برسد، با فشردن شدن مجدد کلید، تاخیر مجدد ۳۰۰ ms خواهد شد.

- شمارش کرونومتر باید به وسیله تایمر انجام شود.
- شمارش کرونومتر بر روی LCD کاراکتری متصل بر روی برد نمایش داده خواهد شد.
- برای تاخیر led، مجاز به استفاده از delay هستید.
- پردازش عدد شمارش شده در تابع وقفه مجاز نمی‌باشد.

مقدمه

در این سوال هدف طراحی یک کرونومتر با استفاده از میکروکنترلر STM32 بود. طبق صورت سؤال، کرونومتر باید با استفاده از تایمر و وقفه کار کند و مقدار آن روی LCD کاراکتری نمایش داده شود. همچنین یک کلید فشاری برای Start/Pause

کرونومتر در نظر گرفته شد که با وقفه خارجی خوانده می‌شود. با هر بار فشردن این کلید، کرونومتر یا شروع به شمارش می‌کند یا متوقف می‌شود.

در کنار کرونومتر، یک LED چشمک‌زن هم طراحی شد که سرعت چشمک‌زدن آن با فشردن کلید دوم تغییر می‌کند. مقدار تأخیر چشمک LED ابتدا ۳۰۰۰ میلی‌ثانیه است و با هر بار فشردن کلید نصف می‌شود؛ یعنی به ترتیب ۳۰۰۰، ۱۵۰۰، ۷۵۰ و ۳۷۵ میلی‌ثانیه می‌شود و بعد دوباره به ۳۰۰۰ میلی‌ثانیه برمی‌گردد.

همچنین مقدار کرونومتر پردازش می‌شود و در صورتی که مقدار ثانیه کرونومتر زوج باشد، بازر یک بوق کوتاه تولید می‌کند. نکته مهم این بود که پردازش زوج بودن داخل تابع وقفه انجام نشود، بنابراین این قسمت در حلقه اصلی برنامه نوشته شد.

در این پروژه کد به صورت رجیستری نوشته شد و از توابع HAL استفاده نشد.

قطعات استفاده‌شده در شبیه‌سازی Proteus

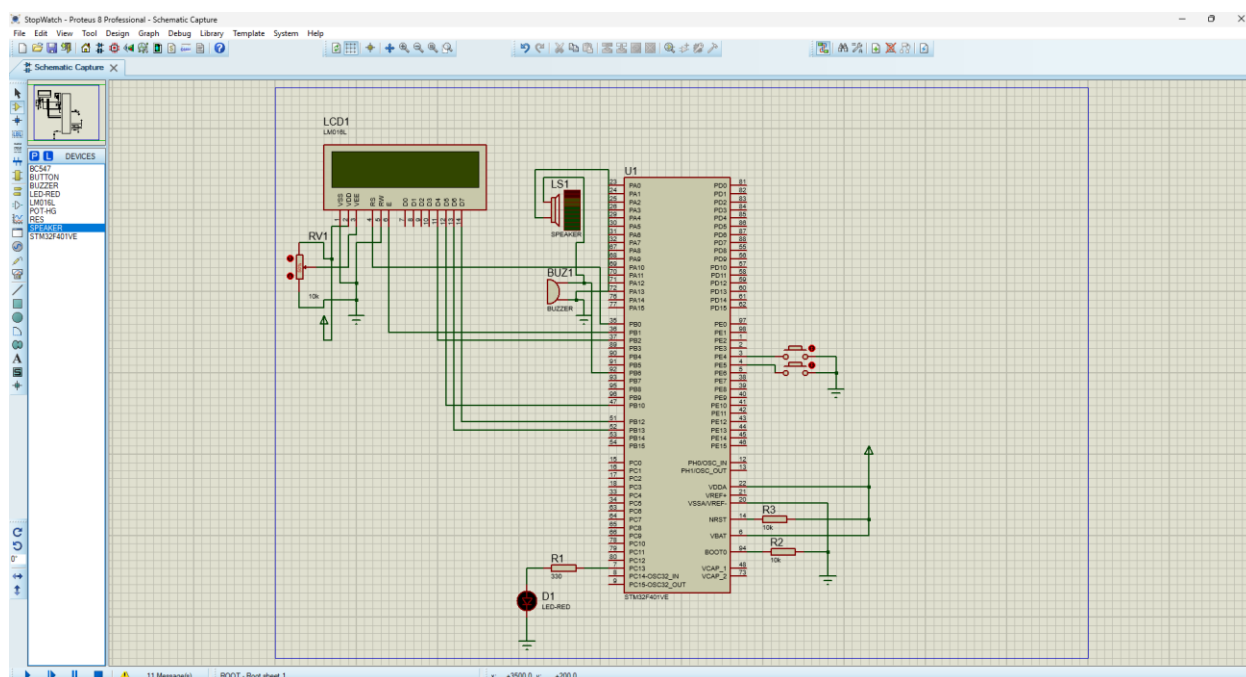
برای شبیه‌سازی مدار از قطعات زیر استفاده کردم:

- میکروکنترلر STM32F401VETx
- LCD کاراکتری ۱۶x2 مدل LM016L
- دو عدد کلید فشاری
- یک عدد LED قرمز
- مقاومت ۳۳۰ اهم برای LED
- بازر یا Speaker
- پتانسیومتر ۱۰k برای تنظیم کنتراست LCD
- مقاومت‌های ۱۰k برای پایه‌های کنترلی مثل Reset و Boot
- منبع تغذیه و زمین

میکروکنترلر انتخاب‌شده STM32F401VETx است، چون هم پورت E دارد و می‌توان برای کلیدها از PE4 و PE5 استفاده کرد، هم پورت C دارد و پایه PC13 برای LED مناسب است. همچنین پایه‌های پورت B برای اتصال LCD استفاده شدند.

اتصال پایه‌ها در Proteus

در این بخش پایه‌های استفاده‌شده در مدار را مشخص کردم. برای ساده‌تر شدن مدار، LCD را در حالت ۴ بیتی وصل کردم، چون در این حالت فقط به ۴ پایه داده نیاز داریم.



جدول اتصال پایه‌ها

توضیح	پایه میکروکنترلر	بخش مدار
ورودی با وقفه خارجی EXTI4	PE4	کلید Start/Pause
ورودی معمولی با Pull-up	PE5	کلید تغییر سرعت LED
خروجی دیجیتال	PC13	LED چشمک‌زن
خروجی دیجیتال	PB6	Buzzer
پایه انتخاب دستور/داده	PB0	LCD RS
پایه Enable	PB1	LCD E
خط داده	PB2	LCD D4
خط داده	PB10	LCD D5
خط داده	PB13	LCD D6
خط داده	PB12	LCD D7

LCD RW	GND	فقط حالت نوشتن
LCD VSS	GND	زمین
LCD VDD	+5V	تغذیه LCD
LCD VEE	وسط پتانسیومتر	تنظیم کنتراست

اتصال کلیدها

دو کلید در مدار استفاده شد.

کلید اول برای Start/Pause کرونومتر است و به پایه PE4 وصل شد. این پایه در برنامه به صورت وقفه خارجی تنظیم شده است. چون برای پایه PE4 مقاومت Pull-up داخلی فعال شده، کلید به صورت زیر بسته شد:

PE4 به یک سر کلید وصل شد و سر دیگر کلید به GND متصل شد. در حالت عادی مقدار ورودی ۱ است و وقتی کلید فشرده می‌شود، مقدار ورودی ۰ می‌شود. به همین دلیل وقفه خارجی روی لبه Falling تنظیم شد.

کلید دوم برای تغییر سرعت LED است و به پایه PE5 وصل شد. این کلید هم با Pull-up داخلی کار می‌کند، یعنی یک سر کلید به PE5 و سر دیگر به GND وصل شده است. این کلید در حلقه اصلی برنامه خوانده می‌شود و وقفه خارجی ندارد.

اتصال LCD

LCD کاراکتری به صورت ۴ بیتی راه‌اندازی شد. در این روش فقط پایه‌های D4 تا D7 LCD به میکرو وصل می‌شوند و پایه‌های D0 تا D3 استفاده نمی‌شوند.

اتصالات LCD به این صورت انجام شد:

- پایه RS به PB0
- پایه E به PB1
- پایه D4 به PB2
- پایه D5 به PB10
- پایه D6 به PB13
- پایه D7 به PB12
- پایه RW به زمین
- پایه VSS به زمین

- پایه VDD به V_{5+}

- پایه VEE به پایه وسط پتانسیومتر $k10$

پتانسیومتر برای تنظیم کنتراست LCD استفاده شد. دو سر کناری پتانسیومتر به V_{5+} و GND وصل شدند و پایه وسط آن به VEE LCD متصل شد. اگر کنتراست درست تنظیم نشود، ممکن است LCD روشن باشد ولی نوشته‌ها دیده نشوند.

اتصال Buzzer و LED

برای LED از پایه PC13 استفاده شد. اتصال LED به این صورت است که پایه PC13 به مقاومت 330Ω و بعد به LED وصل شده و سر دیگر LED به زمین متصل شده است. این مقاومت برای محدود کردن جریان LED استفاده شد.

برای بازر از پایه PB6 استفاده شد. در شبیه‌سازی خروجی PB6 به بازر وصل شد. در حالت واقعی بهتر است برای راه‌اندازی بازر از ترانزیستور استفاده شود، اما در Proteus می‌توان برای تست عملکرد، بازر را به صورت ساده به خروجی میکرو متصل کرد.

تنظیمات CubeMX

ابتدا در CubeMX میکروکنترلر STM32F401VETx انتخاب شد. سپس پایه‌ها مطابق مدار تنظیم شدند.

تنظیم پایه‌ها به صورت زیر بود:

- PE4 به عنوان GPIO_EXTI4 برای کلید Start/Pause

- PE5 به عنوان GPIO_Input برای کلید تغییر سرعت LED

- PC13 به عنوان GPIO_Output برای LED

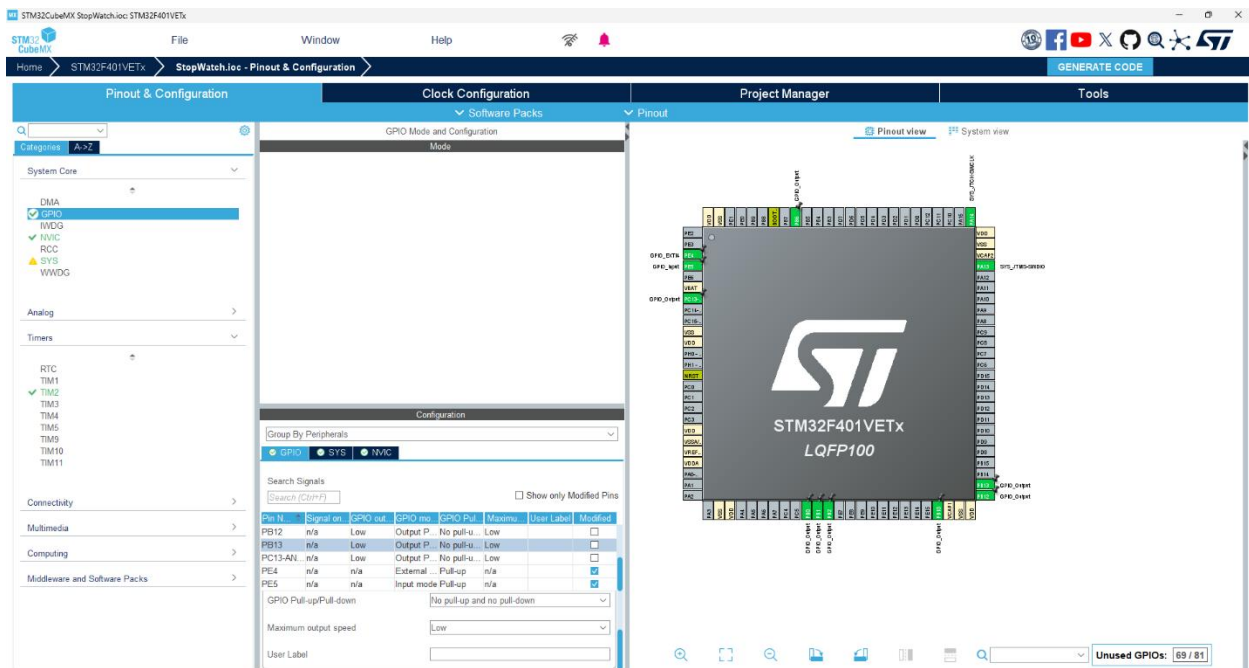
- PB6 به عنوان GPIO_Output برای Buzzer

- PB0، PB1، PB2، PB10، PB12 و PB13 به عنوان GPIO_Output برای LCD

برای PE4 حالت وقفه خارجی روی Falling Edge تنظیم شد، چون کلید به زمین وصل است و با فشردن آن مقدار پایه از ۱ به ۰ تغییر می‌کند.

برای PE4 و PE5 حالت Pull-up فعال شد، چون کلیدها به GND وصل شده‌اند و در حالت عادی باید مقدار ورودی ۱ باشد.

با اینکه پایه‌ها در CubeMX مشخص شدند، برنامه‌نمایی به صورت رجیستری نوشته شد و از HAL استفاده نشد.



تنظیم کلاک

در این پروژه از کلاک داخلی HSI استفاده شد. فرکانس HSI برابر ۱۶ MHz در نظر گرفته شد. به همین دلیل در Proteus هم مقدار Clock Frequency برای میکروکنترلر برابر ۱۶ MHz قرار داده شد.

در تابع Clock_Init ابتدا HSI فعال شد و برنامه منتظر ماند تا HSI آماده شود. سپس مقدار SystemCoreClock برابر ۱۶۰۰۰۰۰۰ قرار داده شد. در این پروژه از PLL استفاده نشد تا تنظیمات ساده‌تر و برای Proteus قابل اطمینان‌تر باشد.

تنظیم تایمر TIM2

برای شمارش کرنومتر از TIM2 استفاده شد. هدف این بود که تایمر هر ۱ میلی‌ثانیه وقفه ایجاد کند.

با توجه به اینکه کلاک ۱۶ MHz است، مقادیر تایمر به این صورت انتخاب شدند:

Prescaler برابر ۱۵ قرار داده شد. بنابراین فرکانس تایمر برابر می‌شود با:

$$16\text{MHz} / (15 + 1) = 1\text{MHz}$$

سپس مقدار ARR برابر ۹۹۹ قرار داده شد. پس تایمر هر ۱۰۰۰ شمارش یک وقفه ایجاد می‌کند:

$$1\text{MHz} / (999 + 1) = 1000\text{Hz}$$

بنابراین زمان هر وقفه برابر ۱ ms می‌شود.

در تابع TIM2_IRQHandler فقط شمارش کروномتر انجام می‌شود. اگر متغیر stopwatch_running برابر ۱ باشد، مقدار stopwatch_ms یک واحد افزایش پیدا می‌کند. بنابراین شمارش اصلی کروномتر با تایمر انجام می‌شود.

استفاده از SysTick

در این برنامه از SysTick هم استفاده شد، اما نه برای شمارش کروномتر. شمارش اصلی کروномتر توسط TIM2 انجام می‌شود. SysTick فقط برای تولید متغیر sys_ms استفاده شد.

متغیر sys_ms برای کارهای جانبی استفاده شد، مثل:

- ایجاد Delay برای LCD
- Debounce کلیدها
- زمان‌بندی چشمک LED
- خاموش کردن بازر بعد از بوق کوتاه
- به‌روزرسانی LCD هر ۵۰ms

بنابراین شرط اصلی صورت سؤال رعایت شد، چون کروномتر با TIM2 شمارش می‌شود و SysTick فقط برای زمان‌بندی‌های کمکی استفاده شده است.

تنظیم وقفه خارجی EXTI4

برای کلید Start/Pause از پایه PE4 استفاده شد. این پایه به EXTI4 متصل شد. در تابع EXTI4_Init ابتدا کلاک SYSCFG فعال شد، سپس EXTI4 به Port E وصل شد. بعد از آن وقفه روی لبه Falling فعال شد.

چون کلید به زمین وصل است و Pull-up داخلی فعال شده، در زمان فشردن کلید لبه Falling ایجاد می‌شود.

در تابع EXTI4_IRQHandler ابتدا بررسی می‌شود که وقفه مربوط به خط ۴ بوده است. سپس فلگ وقفه پاک می‌شود. برای جلوگیری از اثر لرزش کلید، یک Debounce نرم‌افزاری ۲۰۰ میلی‌ثانیه‌ای استفاده شد. اگر از فشردن قبلی کلید حداقل ۲۰۰ms گذشته باشد، وضعیت stopwatch_running تغییر می‌کند. یعنی اگر کروномتر متوقف باشد شروع می‌شود و اگر در حال شمارش باشد متوقف می‌شود.

توضیح متغیرهای مهم برنامه

در برنامه چند متغیر اصلی تعریف شد:

- sys_ms این متغیر با SysTick هر ۱ms افزایش پیدا می‌کند و برای delay و زمان‌بندی‌های کمکی استفاده می‌شود.

- stopwatch_ms
این متغیر مقدار کرونومتر را بر حسب میلی ثانیه نگه می‌دارد و فقط در وقفه TIM2 افزایش پیدا می‌کند.
- stopwatch_running
این متغیر مشخص می‌کند کرونومتر در حال شمارش است یا متوقف شده است.
- led_delay_ms
این متغیر مقدار تأخیر چشمک LED را نگه می‌دارد. مقدار اولیه آن ۳۰۰۰ms است.
- last_led_toggle
این متغیر زمان آخرین تغییر وضعیت LED را نگه می‌دارد.
- last_checked_second
این متغیر برای جلوگیری از بوق تکراری استفاده شد. با استفاده از آن، بازر فقط یک بار در هر ثانیه بوق می‌زند.
- buzzer_active
این متغیر مشخص می‌کند که بازر در حال بوق زدن است یا نه.
- buzzer_off_time
این متغیر زمان خاموش شدن بازر را مشخص می‌کند.

توضیح عملکرد LCD

برای LCD توابع جداگانه نوشته شد LCD. در مد ۴ بیتی راه‌اندازی شد. ابتدا در تابع LCD_Init، دستورهای لازم برای راه‌اندازی LCD ارسال شدند. سپس از تابع LCD_PrintLine برای نوشتن روی هر خط LCD استفاده شد.

روی خط اول LCD زمان کرونومتر نمایش داده می‌شود. قالب نمایش به صورت زیر است:

MM:SS.mmm

یعنی دقیقه، ثانیه و میلی ثانیه نمایش داده می‌شود. همچنین در انتهای خط اول وضعیت RUN یا PAU نشان داده می‌شود. اگر کرونومتر در حال اجرا باشد، RUN نمایش داده می‌شود و اگر متوقف باشد، PAU نمایش داده می‌شود.

روی خط دوم LCD مقدار تأخیر فعلی LED نمایش داده می‌شود. مثلاً:

LED:3000 ms

یا

LED: 375 ms

LCD هر ۵۰ms یک بار به‌روزرسانی می‌شود تا هم مقدار کرونومتر به‌روز باشد و هم برنامه خیلی درگیر نوشتن مداوم روی LCD نشود.

توضیح عملکرد کلید تغییر سرعت LED

کلید دوم به پایه PE5 وصل شده است. این کلید در تابع `Process_Speed_Button` خوانده می‌شود. چون این پایه Pull-up دارد، در حالت عادی مقدار آن ۱ است و هنگام فشردن کلید مقدار آن ۰ می‌شود.

برای این کلید هم `Debounce` نرم‌افزاری ۸۰ms در نظر گرفته شد. وقتی کلید فشرده شود، مقدار `led_delay_ms` تغییر می‌کند. ترتیب تغییر تأخیرها به این صورت است:

3000ms → 1500ms → 750ms → 375ms → 3000ms

در تابع `Process_LED` بررسی می‌شود که آیا از زمان آخرین تغییر وضعیت LED به اندازه `led_delay_ms` گذشته است یا نه. اگر این زمان گذشته باشد، وضعیت پایه PC13 تغییر می‌کند و LED روشن یا خاموش می‌شود.

توضیح عملکرد Buzzer

طبق صورت سؤال باید مقدار کرونومتر پردازش شود و اگر زوج بود، باز یک بوق کوتاه بزند. در این برنامه پردازش زوج بودن داخل وقفه انجام نشده است و در حلقه اصلی و در تابع `Process_Buzzer` انجام می‌شود.

برای اینکه بازر پیوسته و خیلی سریع بوق نزند، مقدار ثانیه کرونومتر بررسی شد. یعنی اگر ثانیه کرونومتر زوج باشد، مثل ۲، ۴، ۶ و ...، بازر یک بوق کوتاه می‌زند.

زمان بوق ۱۰۰ms در نظر گرفته شد. وقتی شرط برقرار شود، پایه PB6 یک می‌شود و بازر فعال می‌شود. سپس زمان خاموش شدن بازر در متغیر `buzzer_off_time` ذخیره می‌شود. بعد از گذشت ۱۰۰ms، بازر خاموش می‌شود.

به این ترتیب پردازش عدد کرونومتر در تابع وقفه انجام نشد و شرط صورت سؤال رعایت شد.

خطاهای Build و اصلاح پروژه

در زمان Build گرفتن، چند خطا ایجاد شد که علت آن خود کد اصلی نبود. مشکل این بود که پروژه در CubeIDE به صورت اشتباه تعداد زیادی فایل اضافی از CMSIS را Build می‌کرد. این فایل‌ها برای این پروژه لازم نبودند و باعث خطاهایی مثل پیدا نشدن فایل‌های زیر می‌شدند:

- `os_tick.h`
- `cmsis_os2.h`
- `RTE_Components.h`
- `arm_nnfunctions.h`
- `arm_nnsupportfunctions.h`

- DAP_config.h

برای رفع این مشکل، فولدرهای اضافی از Build خارج شدند. فولدرهایی که برای این پروژه لازم نبودند عبارت بودند از:

- Drivers/CMSIS/RTOS2

- Drivers/CMSIS/NN

- Drivers/CMSIS/DSP

- Drivers/CMSIS/DAP

- Drivers/CMSIS/Core_A

- Drivers/CMSIS/Core

در واقع برای این پروژه فقط CMSIS اصلی مربوط به STM32 لازم بود، نه RTOS، نه DSP، نه NN و نه DAP. بنابراین این فولدرها یا حذف شدند یا از مسیر Build خارج شدند.

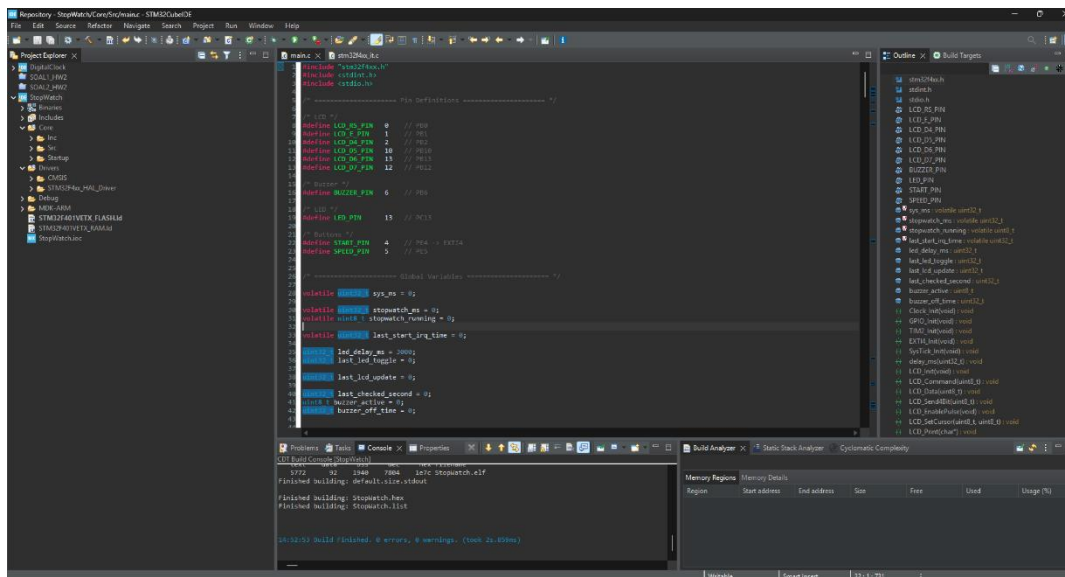
همچنین چون در کد اصلی خودم توابع وقفه را نوشته بودم، اگر همین توابع در فایل stm32f4xx_it.c هم وجود داشته باشند، خطای multiple definition ایجاد می‌شود. به همین دلیل در صورت وجود، توابع زیر از فایل stm32f4xx_it.c حذف یا Comment شدند:

- SysTick_Handler

- TIM2_IRQHandler

- EXTI4_IRQHandler

چون این سه تابع داخل فایل main.c نوشته شده‌اند و برنامه با همان‌ها کار می‌کند.



روند کلی اجرای برنامه

بعد از اجرای برنامه، ابتدا کلاک داخلی فعال می‌شود. سپس SysTick راه‌اندازی می‌شود تا برای delay و زمان‌بندی‌های کمکی استفاده شود. بعد از آن پایه‌های GPIO تنظیم می‌شوند. سپس TIM2 برای وقفه ۱ ms راه‌اندازی می‌شود و EXTI4 برای کلید Start/Pause فعال می‌شود.

بعد از راه‌اندازی LCD، ابتدا پیام اولیه نمایش داده می‌شود. سپس برنامه وارد حلقه اصلی می‌شود.

در حلقه اصلی این کارها انجام می‌شود:

۱. خواندن مقدار فعلی کروномتر

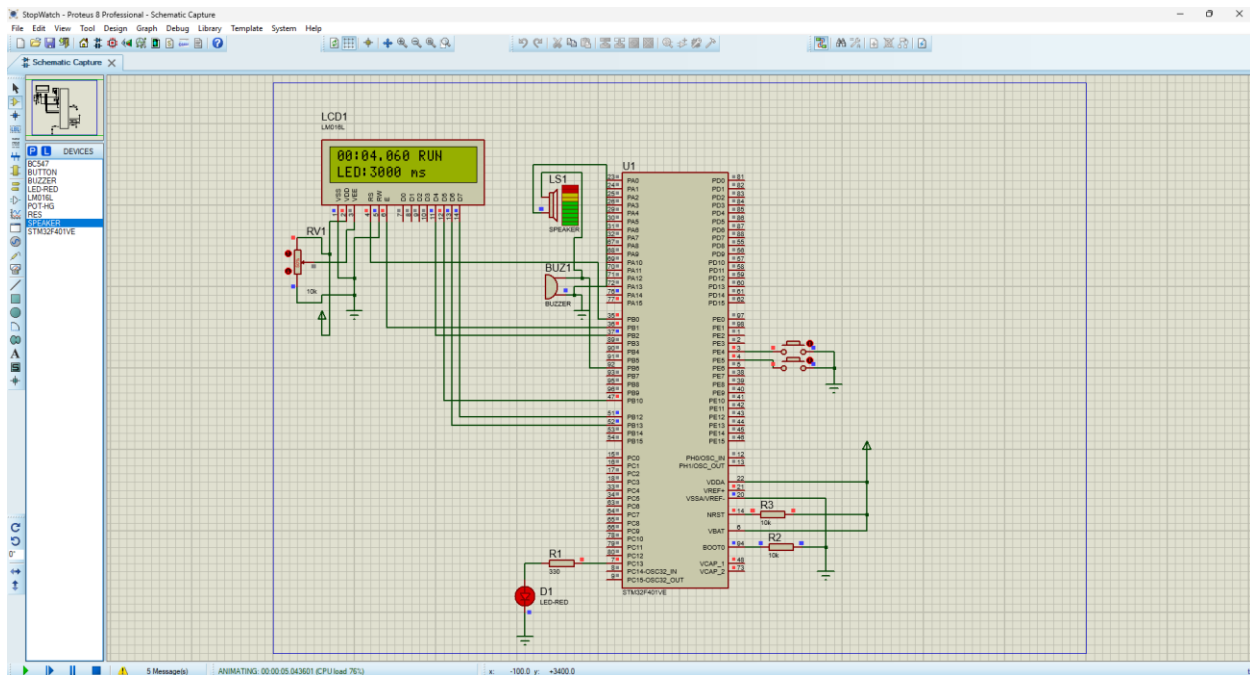
۲. بررسی کلید تغییر سرعت LED

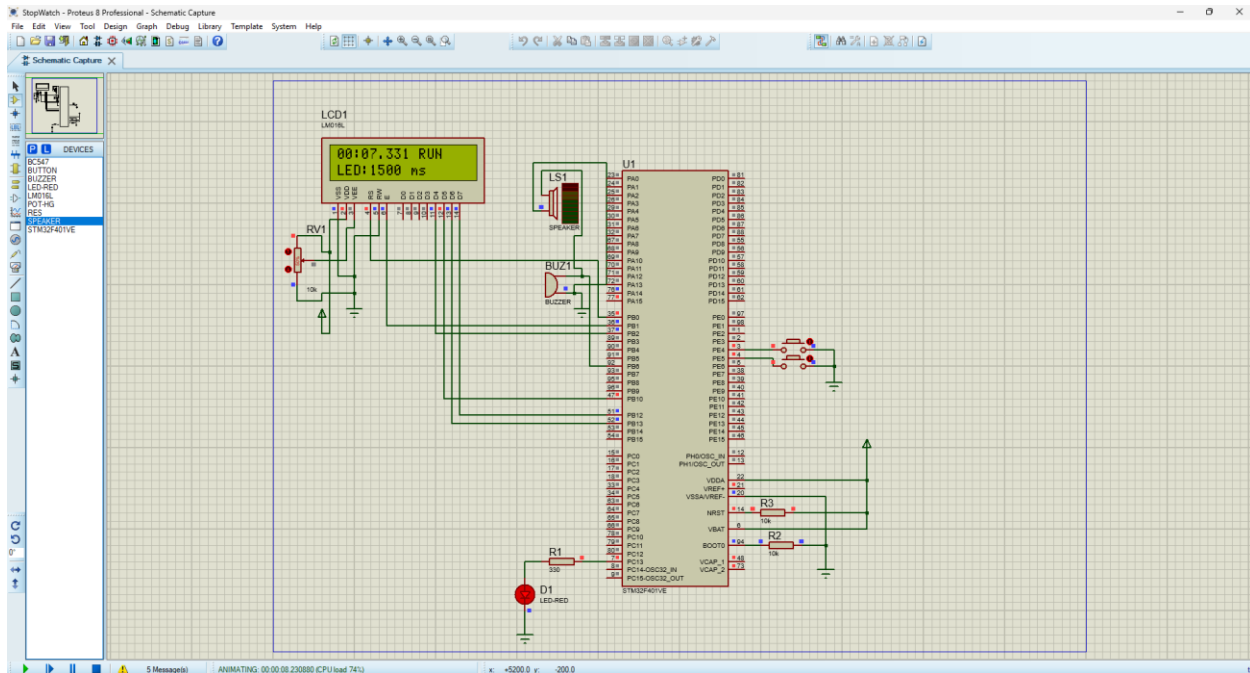
۳. کنترل چشمک LED

۴. بررسی زوج بودن ثانیه کروномتر و کنترل Buzzer

۵. به‌روزرسانی LCD هر ۵۰ ms

در همین زمان، TIM2 در پس‌زمینه هر ۱ ms وقفه ایجاد می‌کند و اگر کروномتر در حالت RUN باشد، مقدار stopwatch_ms افزایش پیدا می‌کند. همچنین با فشردن کلید Start/Pause وقفه EXTI4 اجرا می‌شود و حالت کروномتر تغییر می‌کند.





نتیجه گیری

در این پروژه یک کروномتر با دقت میلی ثانیه طراحی شد که شمارش آن توسط تایمر TIM2 انجام می شود. برای کنترل شروع و توقف کروномتر از کلید فشاری متصل به PE4 استفاده شد که از طریق وقفه خارجی EXTI4 خوانده می شود. مقدار کروномتر روی LCD کاراکتری ۱۶x2 نمایش داده شد و وضعیت اجرای کروномتر نیز با عبارت RUN یا PAU مشخص شد.

همچنین یک LED چشمک زن به پایه PC13 وصل شد که سرعت چشمک زدن آن با کلید دوم تغییر می کند. مقدار تأخیر LED از ۳۰۰۰ms شروع می شود و با هر بار فشردن کلید به ۱۵۰۰ms، ۷۵۰ms و ۳۷۵ms تغییر می کند و سپس دوباره به ۳۰۰۰ms برمی گردد.

برای بازار نیز مقدار ثانیه کروномتر در حلقه اصلی بررسی شد و در ثانیه های زوج یک بوق کوتاه تولید شد. این بخش داخل وقفه انجام نشد تا شرط صورت سؤال رعایت شود.

در نهایت برنامه بدون استفاده از HAL و فقط با استفاده از رجیسترهای میکروکنترلر نوشته شد. همچنین برای رفع خطاهای Build، فایل ها و فولدرهای اضافی CMSIS که برای پروژه لازم نبودند از Build خارج شدند. شبیه سازی در Proteus با میکرو STM32F401VETx، LCD، کلیدها، LED و Buzzer انجام شد و عملکرد کلی مدار مطابق هدف تمرین پیاده سازی شد.